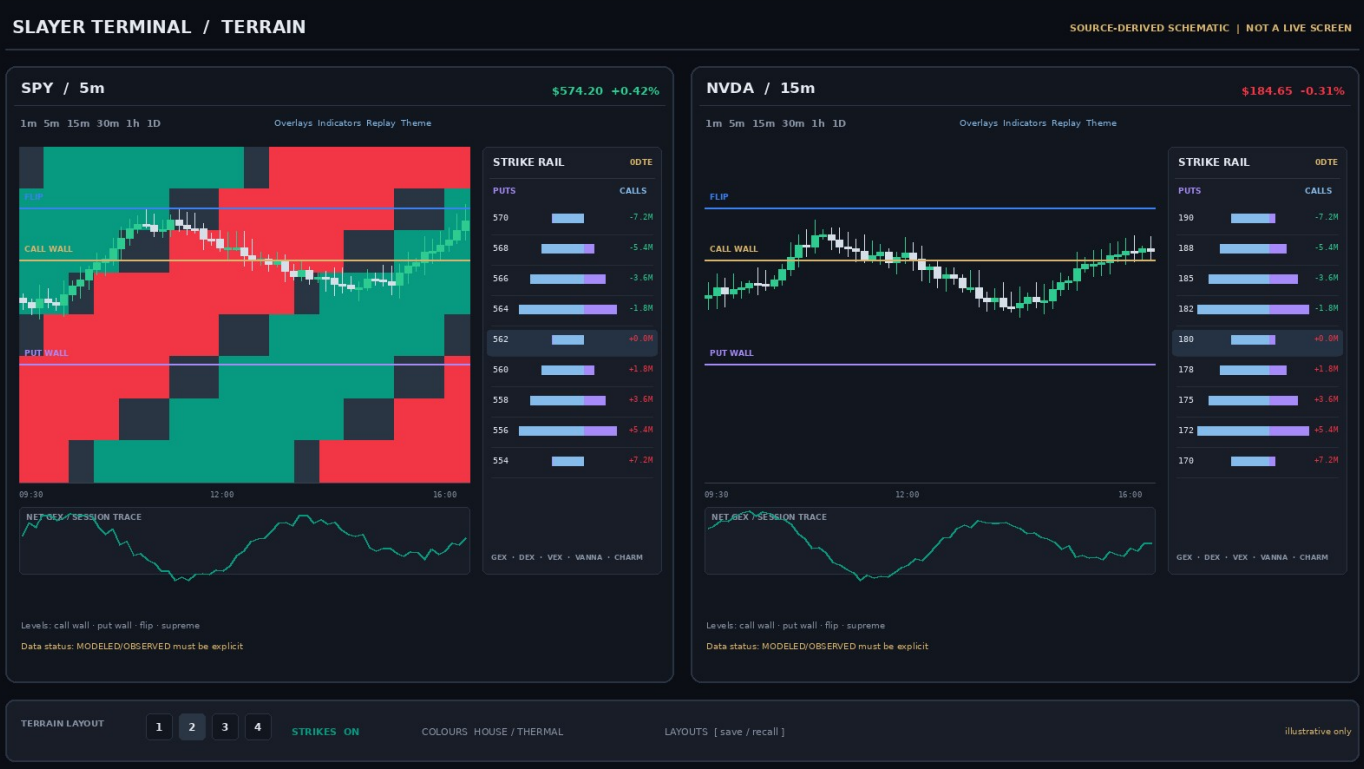
Terrain: The Exposure Workstation

Source audit, OpenMarket cross-reference, and a prioritized plan to make Terrain clearer, more defensible, and more useful to traders.



Source-derived diagram of Terrain's existing concepts. This is an illustrative schematic, not a captured production screen or live market data.

Prepared: October 26, 2026

Repository: hotterthenheat/falcon-cloud-field-sail · main @ b19add2ed3a7

Compared against: user-provided OpenMarket functional teardown (113 pages; 75 screenshots).

Scope: Terrain route, chart indicator registry, exposure calculations, strike rail, expected-move layer, replay, and relevant exposure components.

Purpose: developer execution plan. Existing code findings and proposed work are labeled separately.

01 / DECISION

Build on Terrain’s strengths; fix trust before breadth.

The codebase already has a substantial exposure-chart foundation. The best route to surpassing OpenMarket in this workflow is a coherent, explainable exposure experience - not a race to match the other product’s raw indicator count.

What is already differentiated. Terrain’s code describes a full-screen, one-to-four-pane workspace where each pane owns its symbol, timeframe, overlays, indicators, chart type, comparisons and price scale. An optional strike rail is attached to the pane’s price scale and can read one expiry or the whole book. Exposure logic includes GEX, DEX, VEX, vanna and charm, plus walls, flip, supreme node, pin/OI concentration and friction/air-pocket zones.

Highest-value opportunity. Make every visual read from one explicit data snapshot and one exposure scope per pane. Put the source, timestamp, expiry selection, lens, units and completeness status in front of the user. Then reuse that contract to drive the rail, map, labels, historical migration, alerts and replay.

Priority	Work item	Why it has first-order value
P0 - Trust	Visible source/freshness/completeness, no fake fallback values, one canonical level calculation	Stops a modeled chain, stale surface, or absent flip from appearing to be live certainty.
P1 - Coherence	One Exposure Map with display modes; one scope per pane; map/chart/rail linked by crosshair	Reduces overlap and keeps levels consistent across views.
P1 - Action	Smart levels + alerts + level-migration history + a single readable inspector	Makes the data useful without keeping several large cards over the candles.
P2 - Context	Expected-move calibration, Greek-by-expiry view, verified flow/volume contexts	Adds decision context instead of simply adding more plots.
P3 - Reach	Indicator library, replay auditing, read-only agent/MCP surface	Makes features discoverable and supports repeatable research workflows.

Main recommendation

Do not add every OpenMarket feature as another standalone panel. Unify existing Terrain exposure surfaces, keep the existing per-pane workspace model, and ship provenance + correctness tests before expanding the indicator shelf.

Scope note. Ratings below are design-audit judgments, not user telemetry or benchmark scores. This audit read the repository and your supplied teardown; it did not run your deployment, query your live data vendors, or measure runtime performance.

02 / REPOSITORY AUDIT

Terrain as implemented in source

Snapshot reviewed: main branch commit b19add2ed3a7201599a6b29ec60d1234e0d74849. This section records what code indicates exists today, before recommendations.

Terrain page architecture. `slayer/src/pages/terrain/Terrain.tsx` explicitly states that Terrain is a full-screen chart workspace with one to four panes. Each pane carries its own ticker, tabs, timeframe, overlays, indicator selection, chart style, bar clock, comparison symbols, price scale, opening-range selection, ladder toggle/width, lane mode, theme and link group. The desk also keeps a global rail expiry cut and rail palette. The code intentionally differentiates Terrain from a fixed four-cell board.

Persistence. `terrain/setups.ts` remembers six fields per symbol: timeframe, overlays, indicators, chart style, comparisons and price scale. It validates saved fields individually, uppercases symbols, prevents self-comparison and caps the map at 60 most-recently-touched symbols. `terrain/layouts.ts` saves up to 12 named desk layouts and refuses a thirteenth new layout rather than silently evicting one.

Charting and navigation. `StrikeChart.tsx` includes seven chart styles, multiple price scales, per-pane comparisons, levels/trails, price alerts, session levels, expected-move cone, event markers, flow and volatility bands, and the DEX-by-strike pane. The rail is rendered through `ProfilePanel` and related GEX components. Replay controls are implemented through `ReplayStrip.tsx`.

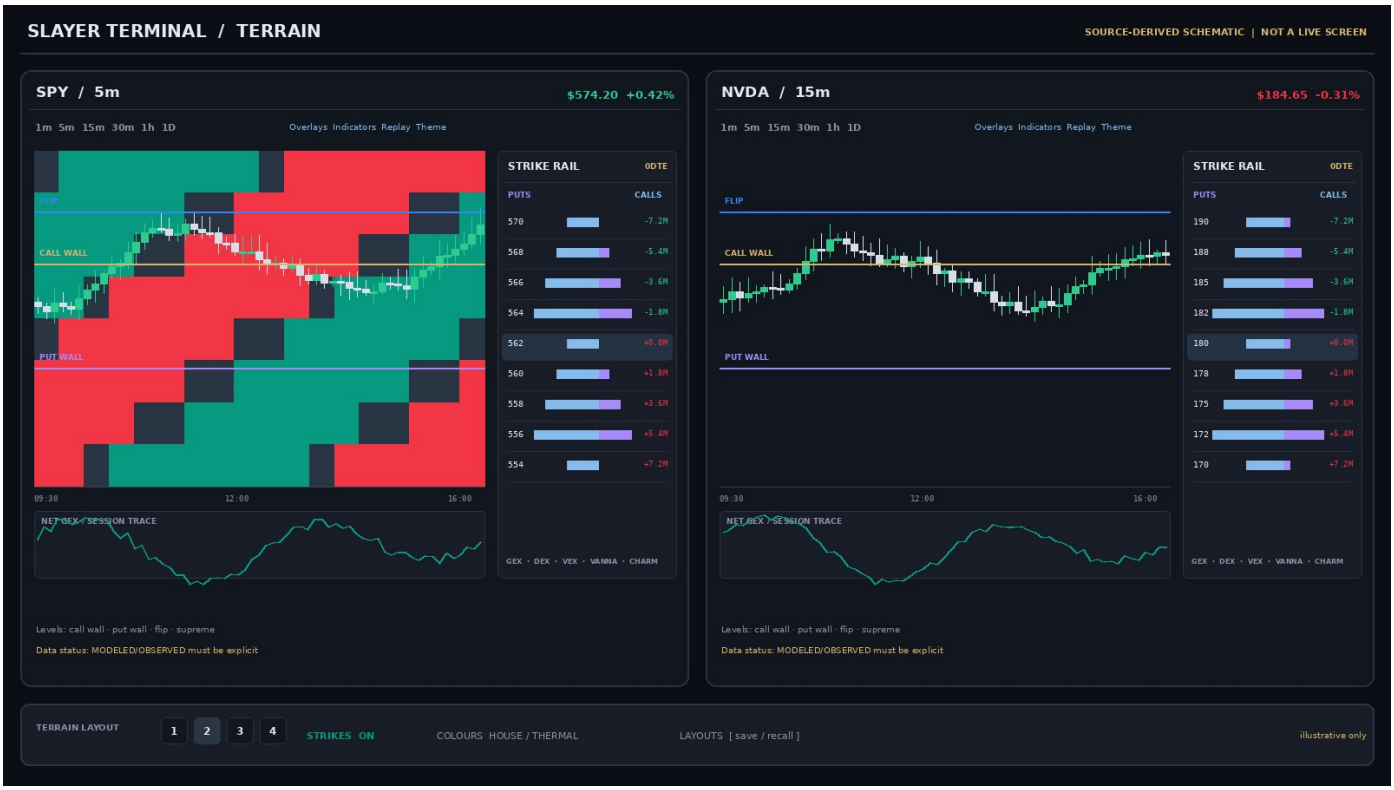
Area	Source files worth preserving / extending	Implementation notes
Terrain shell & pane state	slayer/src/pages/terrain/Terrain.tsx	Per-pane configuration, independent controls, selected expiry cut, layout state, link groups.
Per-symbol setup memory	slayer/src/pages/terrain/setups.ts	Field-level validation; unknown symbols inherit the pane rather than resetting it.
Named layouts	slayer/src/pages/terrain/layouts.ts	Twelve saved layouts; injected validator avoids duplicated schema.
Price indicators	slayer/src/data/indicators.ts; components/gex/indicatorItems.ts	Shared formula generators keep chart calculations and confluence reads aligned.
Exposure calculation	slayer/src/data/exposure.ts; exposureSurface.ts; data/gex.ts	Five Greek lenses, exposure surface, walls/flip/nodes/zones and strike ladder inputs.
Visual exposure layers	components/gex/ExposureField.tsx; gexNodesPrimitive.ts; thermalFieldPrimitive.ts; heatmap.ts	GEX/thermal fields, nodes and heat-map rendering primitives.
Expected move, replay, events	components/gex/expectedMovePrimitive.ts; ReplayStrip.tsx; eventsPrimitive.ts	Existing building blocks for time-aware overlays and review.
Parity / setup tests	slayer/engine/tests/test_exposure_parity.py; slayer/scripts/terrain-setups-proof.ts	Tests are already in place; extend them rather than creating a second rules implementation.

Source links: Terrain.tsx · exposure.ts · exposureSurface.ts · StrikeChart.tsx

03 / WHAT TERRAIN ALREADY DOES

A price chart with its own exposure book

This diagram reconstructs the architecture described by the source: independent panes, local chart controls, price-aligned strike rail, exposure status and a desk-level layout bar. It is intentionally not presented as a screenshot of the deployed page.



Illustrative Terrain source map. Labels and figures are schematic; no live data is shown.

Preserve these behaviors. Keep each pane independently configurable. Keep the rail removable per pane and the rail color choice consistent across the desk. Keep symbol-specific setup memory separate from slot-level layout controls. Keep replay’s historical “where am I?” state visually distinct from live state. These are coherent design decisions already represented in the code comments and types.

Change the way the information is grouped. The strike rail, exposure field, GEX nodes, exposure matrix, and Greek lens should be different views of a shared model, not separate calculations that happen to look similar. One view may be better for reading exposure at a glance; another for comparing calls and puts; another for seeing migration over time.

> **Visual north star**
> Keep the chart as the main object. Move explanatory cards into a docked inspector or compact collapsible legend. Use one shared exposure palette and explicit labels for levels; do not cover candles with stacked cards or make labels compete with price.

04 / CURRENT INDICATORS

What the repository exposes today

The chart’s built-in indicator registry is smaller than OpenMarket’s reported catalog, but Terrain also has a separate exposure/chart layer and user-script path. Do not compare a ten-item price-indicator registry with the other product’s 241-item total as if they were equivalent inventories.

Type	Current keys / controls in source	What to keep / verify
10 built-in indicators	EMA 9, EMA 21, EMA 50, SMA 200, VWAP, VWAP bands, Bollinger Bands, RSI 14, MACD, ATR 14.	Shared pure generators in `data/indicators.ts`; null until warm-up for most oscillators; EMA/VWAP seeded behavior is documented. Show warm-up/data needs in the UI.
12 Terrain overlays	trails, levels, darkpool, volume, flow, netDrift, volDrift, dexStrike, session, cone, events, alerts.	Keep overlays independent and persisted. For each, expose its data dependency, timestamp, supported timeframe and alert availability.
5 exposure Greeks	GEX, DEX, VEX, vanna, charm; net/call/put views and surface by strike and expiry.	Show exact unit, side convention, dealer prior, expiry scope and whether inputs are live or modeled.
Exposure levels	spot, call wall, put wall, flip, pin/OI concentration, supreme, friction and air-pocket zones.	Keep the canonical calculation shared across ladder, map, chips, alert engine and replay. Expose “not found” states honestly.
Chart types	Candles, hollow candles, bars, line, step line, area, baseline.	Consider Heikin-Ashi, volume bars, HLC/columns, volume footprint and TPO only when their source-data and interpretation requirements are satisfied.
Script path	Pine-subset editor and custom-script shelf described by Terrain product page; Slayer namespace exposes exposure to scripts.	Make script inputs, warm-up, required data, execution limits, repaint/look-ahead risk and alert support discoverable.

Existing indicator logic worth keeping centralized. `data/indicators.ts` was separated from chart rendering because confluence and the chart must read the same EMA/VWAP values. RSI, MACD, Bollinger and ATR explicitly encode warm-up windows instead of claiming unsupported early values. Preserve this shared-engine principle for every new indicator.

Source: data/indicators.ts · indicatorItems.ts · StrikeChart.tsx

05 / CROSS-REFERENCE

Terrain vs OpenMarket: capability by capability

OpenMarket side is grounded in the supplied 113-page Functional Teardown, captured October 26, 2026. The report describes the observed UI and public docs; it is not a source-code audit of OpenMarket.

Capability	Terrain - repository evidence	OpenMarket - teardown observation	Developer decision
Workspace	1-4 independent panes, independent controls, per-symbol memory, named layouts.	Up to 16 charts per layout, workspace tabs, crosshair/time/symbol sync options.	Keep 1-4 high-readability Terrain panes; add explicit link modes (symbol, interval, crosshair/time) rather than expanding pane count blindly.
Exposure presentation	Strike rail on chart's price scale; ladder/net lanes; exposure field, nodes, heatmap primitives.	Separate Spot Exposures, GEX profile, GEX curve, Strike Lanes, Heatmap, Matrix and History.	Unify into a single Exposure Map with modes; keep rail as a compact optional view.
Exposures	GEX, DEX, VEX, vanna, charm; walls, flip, supreme, OI/pin candidate, zones.	Greek Exposure offers gamma/delta/vanna/charm/vega/OI/IV×OI; more dashboard tiles and expiration views.	Add small-multiple Greeks and roll-off-by-expiry; expose units and calculation assumptions.
Expected move	Existing thermal cone through `expectedMove.ts` / `expectedMovePrimitive.ts`.	Expected Move cone, separate Odds Cloud, expected-move controls, scorecard proposed in teardown.	Keep cone; add calibrated historical coverage, actual move-used, and horizon selector. Separate forecast bands from probabilities.
Flow / tape	Flow, net drift, vol drift, darkpool, event and largest-print overlay keys.	Flow tab, net premium, top contracts, latest prints, Flow Tape, flow-by-expiry/strike and volatility tabs.	Bring flow to a compact by-strike / time strip; add saved filters and scanner once source coverage is validated.
Indicator discovery	10 built-in indicators plus 12 overlay keys and a custom script path.	241 official indicators in the teardown; 90 cookbook scripts; custom scripting and marketplace.	Build a searchable library with data requirements, previews, units, warm-up, alert support and provenance. Avoid chasing count with redundant indicators.
Alerts	Alerts overlay and broader alert components exist in repo.	Teardown flags missing alerts in several options indicators and recommends an options alert pack.	One alert engine: wall touch/cross, flip change, level migration, flow surge, cone breach and stale data; every alert links to evidence.
Replay	Terrain replay strip and custom bar/history helpers exist.	Replay synchronizes chart/book; teardown notes option-aware replay and replay trading were not confirmed.	Replay exposure snapshots as-of the replay time; never apply today's option book to historical candles. Add read-only audit timeline before replay trading.
Data clarity	Exposure code has live-book and simulator fallback paths; source freshness can be incomplete.	Teardown identifies delay/source badges and inconsistent expiry scope as major UX risks.	Make provenance/freshness/completeness P0 and scope every view from the same pane-local state.

Reference sections in OpenMarket teardown: Options panel and tabs pp. 29-34; exposure/indicator behaviors pp. 35-46; design analysis pp. 89-112. See screenshots on the next pages.

06 / REFERENCE TEARDOWN

OpenMarket’s Spot Exposures: rich but visually costly

The teardown documents a strike-by-time exposure map over the chart, a pinned summary card, key levels, a net-exposure sub-pane, and cell tooltips. It also calls out card clutter, a palette that competes with candles, inconsistent expiry scope, stale stats and missing alert support.

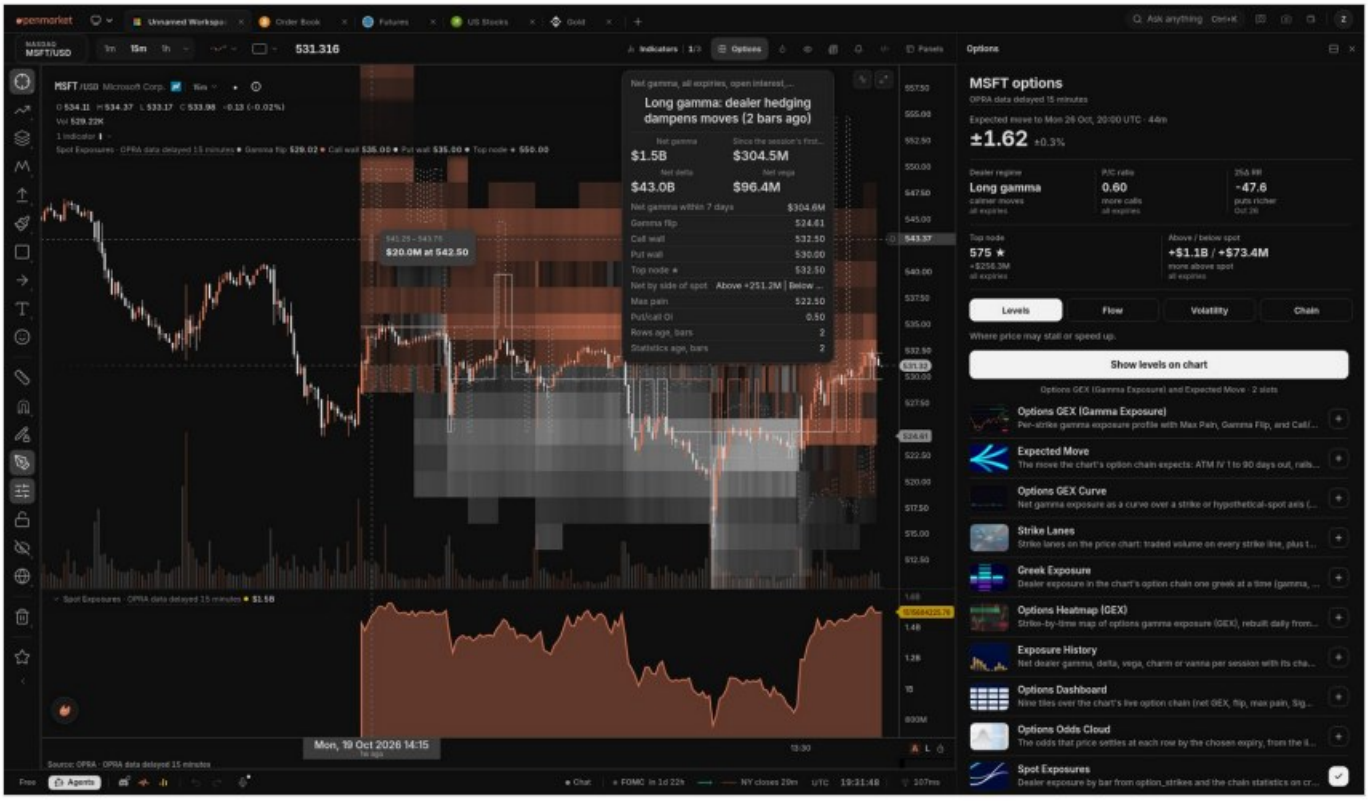


Figure 47. Hovering a cell of the Spot Exposures strike map shows the bucket's dollar exposure.
Reference screenshot crop from the supplied OpenMarket teardown, PDF page 36 (printed page 36; Figure 47). Not Slayer Terminal.

What to borrow. Strike-by-time exposure history, key-level summary, precise hover readouts, multiple lenses, and scope control are valuable. A trader should be able to answer “what is the strongest exposure near spot, how did it change, and how fresh is it?” without opening multiple panels.

What not to copy. The teardown says a large pinned card can cover around 40% of the price action and that the default expiry scope can disagree with other indicators. Terrain has already invested in a rail and a compact pane layout; use a one-line legend plus a docked inspector and a single scope contract to avoid both problems.

> Terrain implementation choice
> Default to a slim inline row: `Net GEX · Flip 577.25 · CW 580 · PW 574 · As of 10:31 ET`. Expand the inspector on click. Keep the complete stats available without keeping a full overlay card pinned over candles.

07 / REFERENCE TEARDOWN

Heatmap plus history: make migration visible

OpenMarket documents a strike-by-time heatmap with volumetric beads or tiles and an Exposure History view with prior-session call wall, put wall, flip, max pain and net exposure. This is the right conceptual area for Terrain’s existing field/node primitives and its surface data.



Figure 56. Options Heatmap (GEX) behind the candles



Figure 57. Options Heatmap settings

Reference screenshot crop from the supplied OpenMarket teardown, PDF page 40. Figure illustrates an exposure heatmap behind price.

Terrain advantage. The repository already has an exposure surface (strike × expiry × Greek), a heat/thermal field renderer, GEX node primitives, strike rails, and historical/replay components. That makes “node migration” an incremental feature if the underlying snapshot series is retained correctly.

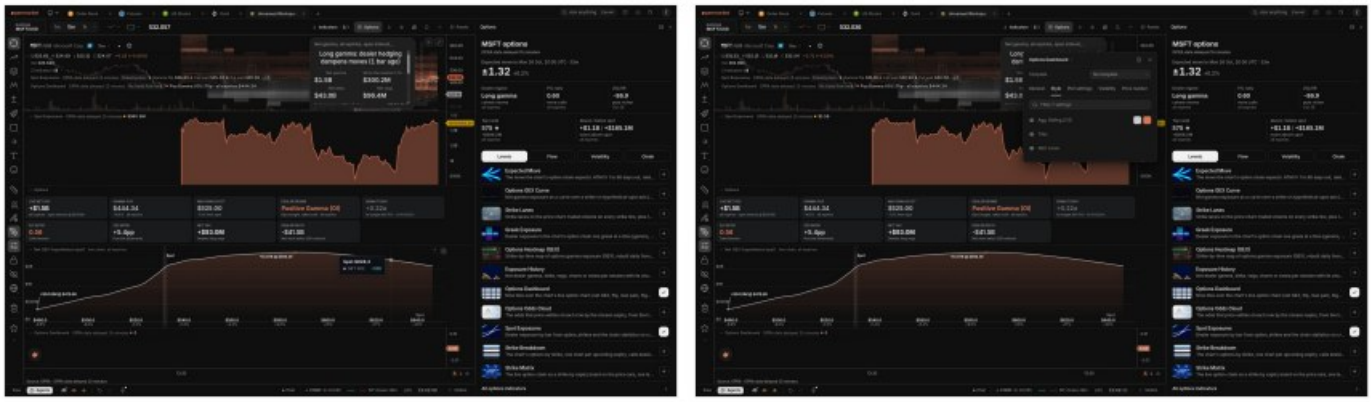
Recommended migration facts. For each node or level, persist the as-of value, price distance, change since open, change vs previous snapshot, and the event that caused a status change. Classify a node as stable/migrating/rolling off only when the time series supports that label. Show the raw previous/current levels behind every summary.

Important. Cell colors are a visualization scale, not a trading probability. If colors are normalized to the visible maximum, display that normalization and units so an apparent brightening is not mistaken for a larger absolute dollar exposure.

08 / REFERENCE TEARDOWN

Dashboard, odds cloud and expected move: combine the useful parts

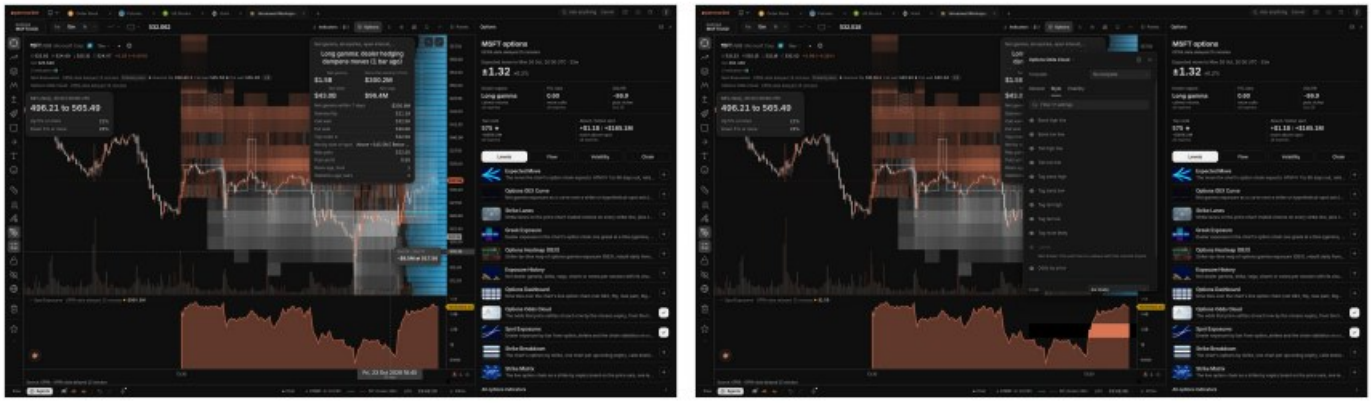
OpenMarket’s teardown documents a dashboard of exposure/volatility metrics, a probability-shaped cloud by price and expiry, and an expected-move cone. It also notes that separate panes can squeeze price and can use differing scopes.



Reference screenshot crop from the supplied OpenMarket teardown, PDF page 41. The original includes options-dashboard tiles and a profile curve.

Terrain already has an expected-move thermal cone. Its renderer draws a forward envelope anchored to the live bar or an appropriate replay/stage bar; the label preserves the claim even when a late-session 0DTE cone collapses. Keep this honesty. Do not widen a near-zero cone just to make it look alive.

What to add. Use a compact dashboard/readout rather than a full-height forced pane: expected move high/low, percentage of the move already used, time remaining, implied-vol source, current spot distance, and rolling historical coverage. If a price distribution or probability cloud is added, validate it against realized data and label it modeled until calibrated.

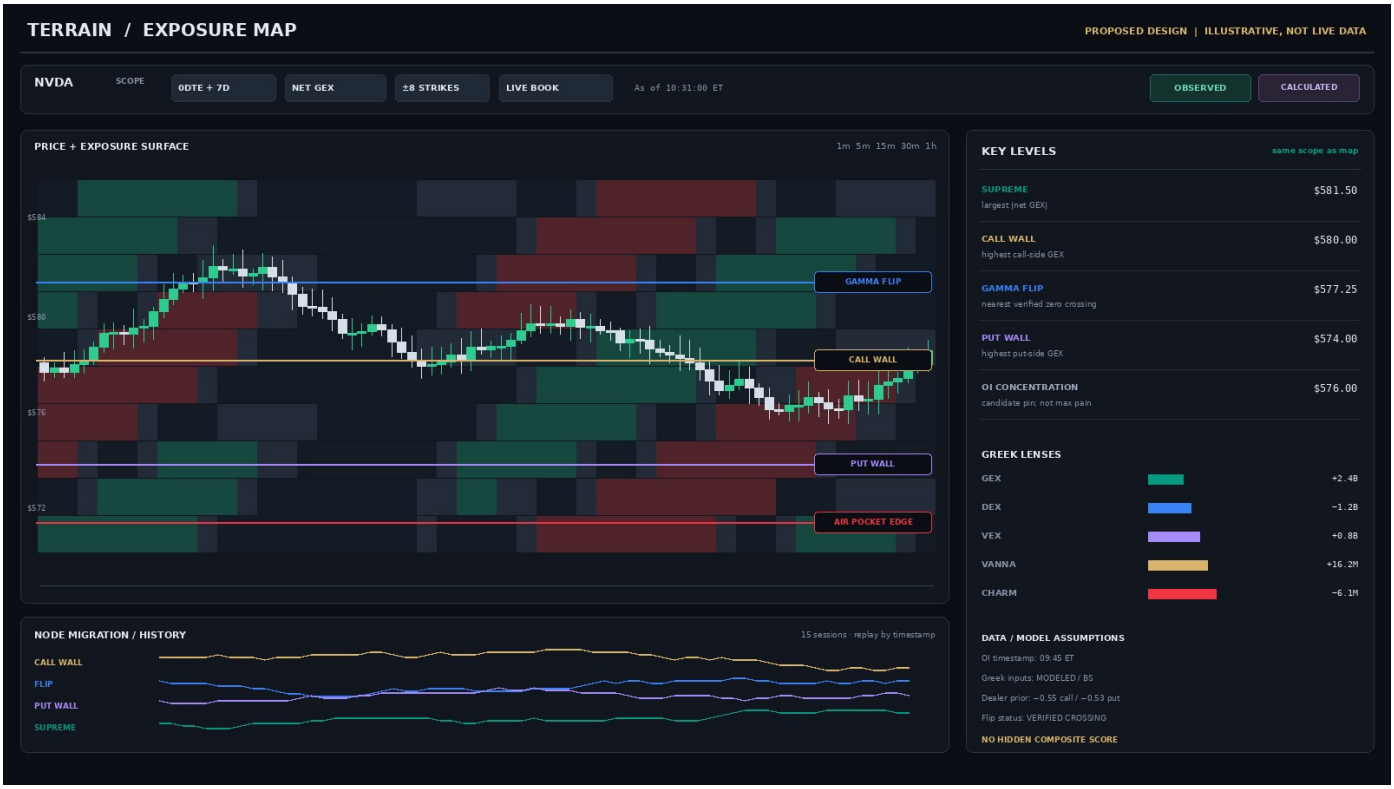


Reference screenshot crop from the supplied OpenMarket teardown, PDF page 42. Odds Cloud is a separate probability-oriented price view.

09 / PROPOSED DESIGN

One Exposure Map, multiple views

This should unify overlapping displays rather than add another independent GEX widget. The scope bar is explicit; each view uses the same snapshot and canonical levels.



Proposed interface concept. All prices and numbers in this figure are illustrative placeholders, not live or calculated market data.

Scope controls - per pane by default: expiry set (0DTE, front, week, month, calendar selection, all), exposure lens (GEX, DEX, VEX, vanna, charm, OI, volume/flow when available), display mode (surface, profile, rail, net history, expiry matrix), strike window, data basis and as-of time. Add an opt-in link icon to link scope across panes. Do not force all panes onto one expiry if they are intended to compare different instruments/horizons.

View modes. Surface = strike × time; Profile = exposure by strike; Ladder = call/put legs and net; Matrix = strike × expiry; History = net exposure and node migration; Greek small multiples = all five Greeks side by side. These are views on a common exposure model, not independent calculators.

Inspector behavior. A selected node opens a right-docked inspector with definition, value/unit, source, time, expiry, call/put split, distance from spot, previous value, and alerts. The inspector collapses; no large persistent card should obscure price.

10 / CALCULATION LOGIC

Single scope, single calculation path

The repo has the right foundation: a chain/profile/surface separation and TypeScript-to-Python parity tests. Extend that design to UI scope and historical snapshots rather than duplicating logic in the Terrain page.

Recommended contract: `ExposureScope = { ticker, asOf, source, sourceStatus, expirySelection, lens, units, strikeWindow, dealerPriorVersion, calculationVersion }`. Build one immutable `MarketSnapshot` and derive the view profile/surface from it. UI components should not re-price contracts or derive walls independently.

```
type ExposureScope = {
  ticker: string;
  asOf: number;                 // exchange/feed timestamp
  source: "OBSERVED" | "CALCULATED" | "MODELED" | "UNAVAILABLE";
  expirySelection: ExpirySelection;
  lens: "gex" | "dex" | "vex" | "vanna" | "charm";
  units: string;
  strikeWindow: number;         // drawing window only
  calculationVersion: string;
};

// One snapshot feeds every view.
const snapshot = buildMarketSnapshot(rawInputs, asOf);
const exposure = buildExposureSurface(snapshot, scope);
const levels = findCanonicalLevels(snapshot.fullChain, scope);
return adaptToMapRailLabelsHistory(exposure, levels);
```

Logic rules. Use the full eligible chain to compute key levels even when the on-screen map is zoomed to a narrow strike window. The current Python engine explicitly describes the window as a drawing choice, not the answer. Keep that principle. Never replace missing inputs with 0. When no flip crossing exists, return an explicit `flipFound: false` state and display “no crossing found” rather than drawing spot as if it were an actual gamma flip.

Units and assumptions: current engine metadata identifies GEX as exposure per 1% move, DEX per 1 sigma move, VEX per 1% vol, vanna per one vol point and charm per one day. Expose those labels and the relevant dealer prior/sign convention with the calculation details.

11 / P0 BEFORE ADDING SIGNALS

Data provenance and fallback modeling

This is the most important technical risk identified in the code review. The UI must not make modeled fallback data look like live exchange/book data.

In `slayer/src/data/exposure.ts`, the profile selects a live chain only when `snapshot.liveBook` is true and the chosen `snapshot.uwChains[expiry]` has rows; otherwise it falls back to `snapshot.chain`. The same source uses deterministic `legJitter` and `strikeVolume` when not in live-book mode. The Python engine's header comments are explicit that its simulator inputs include a seeded book, per-strike jitter, volume hashed from open interest, and modeled farther expiries. Those are valid for simulation/fixtures, but must never be silently represented as observed values.

Required visible provenance

Every key number and visual layer must carry an `OBSERVED`, `CALCULATED`, `MODELED`, `STALE`, or `UNAVAILABLE` status, plus source and timestamp. "Live" describes transport freshness only; it does not make a modeled Greek or inferred dealer position observed. No live signal from an unverified fallback chain.

Build a Data Quality drawer: underlying quote age; option chain timestamp; open-interest date/source; IV source; modeled Greek flag; source entitlement status; missing strikes/expiries; calculation version; last successful refresh; and completeness of full-chain levels. The drawer should be clickable from every visual and be included in exports and replay snapshots.

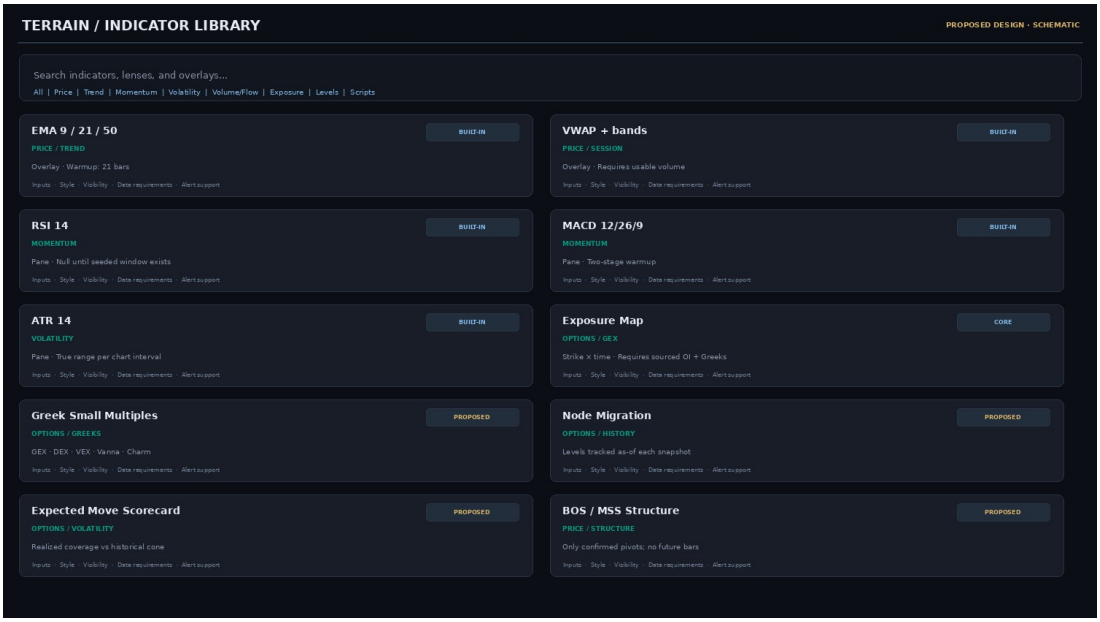
Expose modeling inputs. The Python engine uses dealer prior parameters (-0.55 call and -0.53 put). This is an assumption, not measured dealer inventory. Show the prior and sensitivity band in the explanation panel. Label inferred bias as "model interpretation" rather than "observed dealer positioning."

Never infer truth from fallback values. A spot-level fallback for missing call wall/put wall/flip, a zero default for missing OI, or a placeholder expiry can create a visually convincing but false level. Preserve the associated validity flag all the way to the chart.

12 / INDICATOR ROADMAP

Build a research-grade library, not an inflated catalog

Terrain’s current 10 built-in indicators and 12 overlay keys are a solid base. Expand the catalog by trader question and data requirements, not by duplicating indicators with slightly different plots.



Proposed indicator library concept. Cards/fields are illustrative; the design shows how to surface data requirements and provenance.

Add / upgrade	Why it matters	Required logic / guardrail	Priority
BOS / MSS + swing structure	Adds objective price-action context around levels.	Confirm pivots without using future bars; stamp formation time and confirmation time separately.	P1
EMA alignment / VWAP reclaim	Makes current EMA/VWAP calculations actionable across 1m/5m/15m.	Reuse shared generator; show timeframe matrix and missing-warmup state.	P1
Opening-range / prior-day level package	Makes session structure visible and alertable.	Reuse `sessionLevels` and `OPENING_RANGES`; never silently change session/timezone.	P1
Greek small multiples + expiry roll-off	See whether the exposure concentration is front-expiry or farther out.	Use the same surface/scope; retain true expiry dates; show roll-off percentage only from available OI.	P1
Expected-move scorecard	Tells users whether the cone has historically described realized moves.	Compare predicted band vs realized path, report coverage by horizon/regime/sample count; no unsupported probability badge.	P2
Flow by strike / expiry / time	Connects observed prints to strike exposure and price reaction.	Only use authorized/observed prints; mark side classification as estimated when inferred from bid/ask.	P2
Volume profile / POC / developing VA	Adds price-distribution context to strike and exposure maps.	Requires actual intraday volume distribution; do not substitute OI or synthetic volume.	P2
IV rank / IV-RV / skew surface	Adds premium/volatility context to the expected-move view.	Define lookback, instrument scope, and stale/missing IV behavior.	P2
Footprint / TPO / volume bars	Richer intrabar structure where feed granularity supports it.	Do not invent aggressor side, trades, or volume if only OHLC bars are available.	P3
Indicator script metadata + linting	Makes user scripts safer and discoverable.	Show input schema, warmup, repaint/future-data risk, source fields, permissions, alert support.	P1

13 / NODES AND SMART LEVELS

Make every node explainable and historically traceable

The current exposure engine already calculates a rich set of levels and zones. Improve definitions, consistency and observability before introducing more labels.

Display name	Definition to show in UI	Do not imply
Call wall / put wall	Largest qualifying call-side/put-side exposure on the corresponding side of spot, computed using the canonical level method and the documented expiry scope.	A guaranteed resistance/support level or a price that must reverse.
Gamma flip	Nearest verified zero crossing in the signed net-GEX curve under the current scope. Store whether a crossing exists.	A valid flip when the curve has no zero crossing in the available chain.
Supreme / top node	Strike carrying the largest absolute net-GEX value under the documented universe/expiry scope.	A probability-weighted target.
Pin / OI concentration	High-OI strike or current engine’s specific pin candidate, with the selection rule visible.	“Max pain” unless max pain is separately computed and labeled.
Friction zone	Computed area between relevant levels when width/spacing criteria pass.	A low-volatility guarantee.
Air pocket	Region meeting the canonical low-exposure/spacing definition. Show width and exposure context.	A guaranteed fast-move prediction.
Node migration	Current level vs prior snapshots, delta, elapsed time, expiry roll-off and source completeness.	A directional signal unless tested and explicitly specified as one.

Recommended level interaction. Hover = exact level definition, value, unit and time. Click = pin to chart and open inspector. Right-click = add alert, copy level, inspect history, or hide. On chart, compact labels should show short name + price + movement from open; full details stay in the inspector.

Smart levels. A call wall, flip, opening-range boundary or expected-move edge can be a live line, but every line must be linked to its source and have a history. If it moves, store the prior and new values; the chart should not silently slide a line and erase why the user drew attention to it.

14 / ALERTS AND EVENT PIPELINE

One alert engine, many evidence-backed conditions

OpenMarket’s teardown calls out alert support gaps across several exposure indicators. Terrain should use its existing alerts components but centralize triggers and presentation so the same condition does not fire differently from different panels.

Alert family	Trigger	Payload / safety
Key level	Price touches/crosses call wall, put wall, flip, supreme or selected smart level.	Exact as-of level value, current spot, expiry scope, cross direction, data age.
Node migration	Level moved more than threshold over selected time window or changed rank.	Prior/current level, absolute and percentage shift, snapshots used, sample interval.
Exposure regime	Canonical net GEX or verified flip condition changes classification.	Raw net value, threshold, convention, source/model state, prior regime.
Expected move	Spot reaches 1σ/2σ boundary, or actual move use exceeds defined fraction of the range.	Cone as-of time, horizon, IV source, actual move and selected band.
Flow	Qualifying options prints/net premium exceed saved criteria.	Source feed, print timestamps, size, trade-side classification and confidence/quality flag.
Structure	Confirmed break of opening range / prior day high-low, BOS/MSS, VWAP reclaim.	Rule inputs, timeframe, confirmed candle, invalidation and event timestamp.
Data integrity	Feed stale, OI/chain missing, expiry mismatch, no valid flip, model fallback active.	This should be a system alert, not a trade signal.

Alert rules: deduplicate by `(user, symbol, condition, level/version, timeframe)`; implement per-rule cooldown; support test-fire; store a complete event log; show exactly why an alert fired. Push/webhook delivery depends on enabled integrations and permissions. Do not use a natural-language alert builder to invent rules silently: first convert the requested sentence into a visible structured rule and require review.

15 / EXPECTED MOVE AND REPLAY

Make history honest: calculate each point as-of that moment

The Terrain code already contains a custom expected-move thermal layer and replay transport. Use them to build validation, not to paint today’s exposure over yesterday’s prices.

Expected move upgrade. At each timestamp, persist the chain/IV inputs used to build the cone, anchor time, horizon, upper/lower bands and source status. Add a scorecard by symbol and horizon: number of comparable sessions, percent of actual paths inside each band, median absolute error, and breakdown by volatility regime. Show the sample size and label small/unstable samples. Do not call a 1σ band “68% likely” unless the model is empirically calibrated for that horizon and product.

Replay upgrade. For every replay step, reload the market snapshot and exposure surface from the historical as-of time, then recompute levels from the information available then. The replay clock should govern exposure, flow, events and alerts alike. If historical open interest or option chains are not available for that timestamp, show `UNAVAILABLE` or `MODELED - historical chain unavailable`; do not substitute current data.

Replay audit timeline. Record event time, source timestamp, model/calculation version, value before/after, alert evaluation and UI action. Support export as JSON/CSV for research review. The first version can remain read-only; simulated entries and P&L; can follow after quote replay is trustworthy.

> Critical acceptance test
> Move the replay cursor to yesterday 10:05 ET. Every level, cone, node, flow summary and alert state must be calculated from 10:05 ET-available inputs, not today’s book or later rows.

16 / CODE-LEVEL IMPLEMENTATION

Use the existing abstractions; avoid a second exposure engine

These snippets are abbreviated from current files to show the architectural seam. The implementation should edit the actual files and keep the existing naming/contracts; do not paste these fragments as a replacement for full modules.

A. Current built-in indicator registry - extend the registry and pure series functions together. Keep each indicator key in `StrikeChart`, its label/hint in `indicatorItems.ts`, setup validation in `terrain/setups.ts`, and tests in `data/indicators.test.ts` or the current test pattern.

```
export const INDICATOR_ITEMS: IndicatorItem[] = [
  { key: 'ema9', label: 'EMA 9', hint: '9-bar exponential moving average' },
  { key: 'ema21', label: 'EMA 21', hint: '21-bar exponential moving average' },
  { key: 'ema50', label: 'EMA 50', hint: '50-bar exponential moving average' },
  { key: 'sma', label: 'SMA 200', hint: 'The long classic the EMA trio does not cover' },
  { key: 'vwap', label: 'VWAP', hint: 'Volume-weighted average price, session-anchored' },
  // ... VWAP bands, Bollinger, RSI, MACD, ATR
];
```

B. Overlay contract already exposes useful panes - keep each overlay independent and persisted, but add `dataNeeds`, `status`, `units`, `alertSupport`, and `warmup` metadata in a registry so the UI can explain it.

```
export interface ChartOverlays {
  trails: boolean; levels: boolean; darkpool: boolean; volume: boolean;
  flow: boolean; netDrift: boolean; volDrift: boolean; dexStrike: boolean;
  session: boolean; cone: boolean; events: boolean; alerts?: boolean;
}
```

C. Exposure input fallback must be represented in UI state - current profile code selects a live chain conditionally and uses `snapshot.chain` as fallback. Keep the fallback if needed for demo/test, but pass the provenance flag through to every view.

```
const liveChain = snapshot.liveBook ? snapshot.uwChains?.[expiry] : undefined;
const chain = liveChain && liveChain.length ? liveChain : snapshot.chain;
const factor = snapshot.liveBook ? 1 : EXPIRY_DECAY[expiry];
// Surface and levels must also return sourceStatus + complete + valid flags.
```

D. Use one canonical scope to drive all views - proposed integration shape, not current API.

```
const scoped = selectExposureScope(snapshot, pane.exposureScope);
const profile = buildExposureProfile(scoped.fullChain, scoped.spot, scoped.window);
const surface = buildExposureSurface(scoped, scoped.window, scoped.expiries);
const levels = buildCanonicalLevels(scoped.fullChain, scoped.spot);
renderMap(surface, levels, scoped);
renderRail(profile, levels, scoped);
renderLevelLabels(levels, scoped);
registerAlerts(levels, scoped);
```

17 / ACCEPTANCE TEST PLAN

Proof required before the new layers are called complete

The repository already has TypeScript/Python exposure parity testing. Extend those fixtures and add UI-level acceptance tests for state consistency and data truth.

Test	Scenario	Pass condition
Single-scope parity	Same ticker, as-of time, expiry set and Greek lens in map, rail, summary chip and alert preview.	Numerically identical levels/units; same source status and timestamp.
Window independence	Change the rendered strike window while keeping scope fixed.	Visible map changes; canonical full-chain walls/flip/supreme do not change merely because the chart zoomed.
No flip found	No zero crossing in the full eligible curve.	No fake flip line at spot; show `NO CROSSING FOUND` and disable flip-cross alert.
Fallback data	Live book flag false or live chain missing.	Every exposure view visibly marks MODELED/FALLBACK; no `LIVE` state or trade alert pretending to use observed data.
Partial data	One expiry missing OI, IV or Greeks.	Only affected fields become UNAVAILABLE; no zeros inserted; scope completeness degrades.
Stale data	Chain older than agreed TTL while spot updates.	Source age shown; sensitive exposure alerts pause or enter an explicit stale state.
Replay as-of	Replay a historic bar while newer chain snapshots exist.	No future snapshot leaks into that replay frame; migration and cone rebuild by historical timestamp.
Pane independence	Change symbol, timeframe, theme, rail visibility and indicators in pane A.	Pane B remains unchanged unless the matching link option is intentionally enabled.
Setup migration	Load a stored layout with a retired indicator/overlay key.	Valid keys survive; unknown keys drop safely; no whole setup resets.
Accessibility	Keyboard-only use and screen-reader announcement.	Scope controls, rail, alert actions, menus and replay have visible focus and accessible labels.
Performance	One, two and four panes with map/rail active.	No hidden panes mounted unnecessarily; updates do not thrash layout; panel state remains responsive.
Snapshot audit	Export a signal/alert or replay frame.	Includes input timestamps, source, units, scope, calculation version, raw level values and reason.

Tests already present: `slayer/engine/tests/test_exposure_parity.py` compares chain, profile, surface, walls/flip and air-pocket outputs to a TypeScript reference fixture. `slayer/scripts/terrain-setups-proof.ts` checks symbol memory, field-level validation, self-comparison rejection, cap/eviction and round-trip setup persistence. Keep those proofs and extend them at the shared-scope boundary.

18 / DELIVERY PLAN

Ship in gates, with correctness ahead of new indicators

Recommended order: first a small, auditable data-contract change; then a unified view; then actionable history/alerts; then new indicators. This minimizes risk of adding UI layers before the model can support their claims.

Phase	Deliverables	Gate to proceed
Phase 0 - Trust (first)	Data provenance enum; source timestamps; `complete`/`flipFound` and per-field availability; selected scope; canonical level function shared by the rail and map; no fake fallback lines.	Fixtures show parity for current calculations. UI visibly distinguishes live, modeled, stale and unavailable.
Phase 1 - Unified map	One scope bar per pane; Profile / Ladder / Surface / Matrix / History modes; compact inline level legend; right inspector; crosshair highlights selected strike.	Changing scope updates all components in the pane together; peers stay independent by default.
Phase 2 - Monitoring	Smart level events, alerts/cooldowns, node migration, timestamped event log, export; alert preview links to exact source snapshot.	Replay and live use the same event schema; alerts can be replayed from logs without duplicate events.
Phase 3 - Research context	Expected-move scorecard, Greek-by-expiry small multiples, validated flow/by-strike context, structure overlays and indicator registry metadata.	Every indicator has source requirements, units, timeframe, warmup and a test; no implied probability without calibration.
Phase 4 - Ecosystem	Script gallery previews/metadata, presets, read-only MCP/agent methods for snapshots/levels/alerts; later simulation/replay orders if licensed data permits.	Permissions are explicit; read-only default; user data and vendor entitlements are respected.

Do not do this first

Do not add a huge list of technical indicators, a 0-100 “Terrain score”, AI-generated explanations, or a simulated future-candle overlay before data provenance and historical scope replay are correct.

19 / COPY-READY DEVELOPER BRIEF

Implementation request

Give this section to the developer as the summary task. The rest of the PDF explains why each requirement exists and how to validate it.

```
GOAL
Upgrade Terrain's exposure workflow from a collection of capable chart/rail components into one consistent, explainable,
  timestamped exposure workstation. Extend the existing code; do not replace the independent pane model.

REQUIRED
1. Create a pane-local ExposureScope: ticker, as-of timestamp, source status, expiry set, Greek lens, units, strike window,
  source and calculation version. Optional link control may synchronize scope across panes.
2. Build one snapshot and one canonical set of levels per scope. Map, rail, level chips, inspector, alert engine and replay must
  consume the same outputs.
3. Preserve the 5-Greek engine, full-chain level logic, rail, GEX nodes, thermal/expected-move overlay, price-action overlays,
  replay, per-symbol memory and named layouts.
4. Make fallback/simulator chains visible as MODELED. Missing OI/IV/flip must remain UNAVAILABLE / NOT FOUND, never zero or spot-
  level truth.
5. Add Exposure Map modes: Surface, Profile, Ladder, Strike x Expiry Matrix, History, Greek Small Multiples. Keep the UI compact
  and keep cards out of the candle area.
6. Add node/level migration, data-quality status, one alert engine, as-of replay, and an indicator gallery with metadata.
7. Keep 10 existing built-in indicators and 12 overlay keys, but make source requirements, units, warmup, timeframe and alert
  support discoverable. Add new indicators only where data is available.
8. Extend existing parity/setup tests; add no-flip, stale-data, missing-OI, snapshot-scope, replay-look-ahead, and pane-
  independence acceptance tests.

NO-GO
No invented market data, no silent fallback to current data during replay, no opaque composite score, no “dealer inventory
  observed” language when inferred from OI and a fixed prior, no unsupported probabilities, and no external-data redistribution
  unless licensing permits it.

DONE WHEN
The same scope yields the same levels and units in every view; provenance is visible; the history is reproducible; alerts point
  to their evidence; and 1/2/4-pane layouts stay responsive without changing unrelated panes.
```

Primary technical files: Terrain.tsx · setups.ts · layouts.ts · StrikeChart.tsx · exposure.ts · exposureSurface.ts · indicators.ts · Python exposure engine · parity tests

20 / APPENDIX A

Indicator library schema

Recommended metadata for every built-in indicator, overlay, exposure lens and user script. This is what enables search, consistent UI, test generation and honest data state.

```
type IndicatorDefinition = {
  id: string; label: string; category: string;
  description: string; displayType: 'overlay' | 'pane' | 'rail' | 'surface' | 'event';
  inputSchema: InputDefinition[];
  inputs: DataRequirement[];
  warmupBars?: number;
  supportedTimeframes: string[];
  units?: string;
  dataStatus: 'OBSERVED' | 'CALCULATED' | 'MODELED' | 'UNAVAILABLE';
  sourcePolicy: 'required-live' | 'historical-ok' | 'model-permitted';
  alertSupport: boolean;
  replaySupport: boolean;
  repaintRisk: 'none' | 'pivot-confirmation' | 'script-defined';
  tests: string[];
};
```

Library behaviors. Search by name/category/market question. Preview the actual line/surface on a sample, but label previews as demo if data is not real. Show required inputs before adding. Use a settings drawer with Inputs, Style, Visibility, Data Requirements and Alerts tabs. Show “not drawn: missing values” instead of quietly dropping a line. Keep the classic indicators unlimited from a computation standpoint; data-gated overlays should be governed by vendor entitlements and load budget, not mystery UI slot counts.

Suggested categories: Price & trend; momentum; volatility; volume & flow; market structure; session levels; options exposure; options volatility; data health; scripts. Favourites/recent and pane presets should exist, but presets must document every included calculation.

21 / APPENDIX B

Evidence map and reading links

Findings below are traceable to repository paths or to pages in the user-provided OpenMarket teardown. OpenMarket descriptions are source-derived observations, not assumptions about its private implementation.

Evidence	What it supports
Terrain.tsx	1-4 panes; per-pane state; rail expiry cut/palette; pane/chart composition; keyboard, layout and replay plumbing.
setups.ts / terrain-setups-proof.ts	Symbol setup fields; schema validation; field-level restore; 60-symbol cap and round-trip checks.
StrikeChart.tsx / indicatorItems.ts	Chart overlay keys, seven chart styles and 10 built-in indicator definitions.
exposure.ts / exposureSurface.ts	Five Greek exposures, OI concentration/pin logic, walls/flip/zones, live/fallback branches and strike×expiry×Greek surface.
slayer_core/exposure.py / test_exposure_parity.py	Dealer prior notes, simulator input texture, output contracts, and Python/TypeScript parity testing.
expectedMovePrimitive.ts / ReplayStrip.tsx	Expected move field anchoring/labels and replay transport semantics.
OpenMarket Teardown, pp. 29-34	Options header metrics, tabs and chain; pp. 35-46 detailed GEX/flow/volatility indicators.
OpenMarket Teardown, pp. 99-112	UI critiques and ranked product blueprint: unify scope, key-level sheet, smart levels, merged map, docked cards, alerts, replay, calibration and data-status badges.

Known limitation. This brief is based on code and assets accessible through the public repository snapshot at the stated commit and the supplied OpenMarket PDF. It does not constitute a live production QA run, a vendor-data licensing opinion, or a claim that OpenMarket's calculations are ground truth. The PDF uses source-derived diagrams and clearly captioned reference screenshots; conceptual mockups are not actual production screenshots.

Finish line

Terrain should be better at explaining the source, scope, and evolution of its own exposure levels than any fragmented indicator stack. The differentiator is one coherent model that can be trusted, inspected, replayed and acted on - not simply more lines on the chart.